

Agent GitHub 使用案例

Agent GitHub 主要解决三个场景。

第一，本地多 Agent 同步。

一个人在本地开发时，常常不会只开一个 Agent。比如一个 Agent 在读代码，一个 Agent 在改实现，另一个 Agent 在查文档。问题是它们彼此不知道对方做了什么。读代码的 Agent 找到了关键模块，写代码的 Agent 还在重复搜索；查文档的 Agent 已经确认了 API 行为，review Agent 仍然按旧理解判断。

所以需要本地同步。最简单的是单向同步：把 research Agent 的结论同步给 coding Agent。更进一步是双向同步：coding Agent 的实现进展也同步回 research Agent，让双方都知道当前状态。轻量版可以先用规则做，比如按文件、todo、summary 同步；复杂版本可以让模型判断哪些信息该同步、哪些冲突需要保留。

第二，多人共享 Agent。

在同一个项目里，不同人会有自己的 Agent。比如 Alice 的 Agent 做了调研，Bob 的 Agent 写了实现，Carol 的 Agent 做了 review。现在的问题是：这些 Agent 的上下文只在各自手里。别人想接着做，就要先问人、翻聊天记录、重新解释背景。

Agent GitHub 要让这些 Agent 像代码分支一样可上传、下载、checkout、merge。Bob 可以拉取 Alice 的 Agent context，直接知道调研结论；Carol 可以 merge Bob 的实现上下文，接着 review。多人协作里一定会出现冲突，比如一个 Agent 认为接口字段叫 `user_id`，另一个认为叫 `uid`。这时不能简单覆盖，要像 Git Merge 一样把冲突显式列出来，交给别人判断。

第三，break Agent 和 Project 的互相索引关系。

现在很多 Agent 默认绑定在某个 Project 里。这个设计会限制复用。真实工作里，Agent 往往不是项目的一部分，而是一种能力。Project 是任务边界，Agent 是能力资产。

比如一个系统拆成前端项目和后端项目。前端和后端是两个 Project，但它们需要共享同一个 API Agent、Docs Agent、Deploy Agent。API Agent 不应该只属于前端或后端，它应该能同时被两个 Project 索引和调用。反过来，一个 Project 也不应该只能看见自己的 Agent，它可以引用外部已有 Agent。

这就是 break Agent 和 Project 的互相索引关系：Agent 可以跨 Project 存在，Project 也可以引用多个外部 Agent。这样团队不用每个项目重新养一套 Agent，而是把成熟 Agent 当作公共资产复用。

(注：内容由 AI 生成，请谨慎参考)